Student Record System — Software Architecture

1. Purpose

This document describes the proposed architecture before coding: system layers, component responsibilities, data flow, and how requirements are supported. The specific programming language, framework, and database are not assumed; the team should confirm them before implementation.

2. Architecture Style

Use a three-tier client–server architecture: Presentation Layer, Application/Business Logic Layer, and Data Layer. Separating these responsibilities supports testing, maintenance, and controlled access to academic records.

3.1 Presentation Layer

A web interface for Students, Faculty, and Administrators. It provides login/logout, attendance, marks entry, academic records, student search, reports, notifications, and downloads. The interface shows only the functions appropriate to the signed-in user's role.

3.2 Application Layer

Processes authentication and authorization, validates inputs, records attendance, calculates attendance percentage, manages marks and subjects, converts marks to grade points, calculates semester GPA and CGPA, generates reports, sends low-attendance notifications, and prepares downloads. Permissions and validation must be enforced here, not only in the UI.

3.3 Data Layer

Stores user accounts and roles, student profiles, subjects and credits, attendance, assessment marks, academic results, and notification/report metadata. Only the application layer accesses the database; the browser must never connect directly to it.

3.4 Supporting Services

Logging and error handling, backup and recovery, and automated repository checks support availability, data integrity, and maintainability.

4. Logical Architecture Diagram

```
Student / Faculty / Administrator
              ↓
Presentation Layer – Web UI
              ↓ HTTPS / API requests
Application Layer – Authentication, Authorization, Validation, Business Logic
              ↓ Controlled database access
```

```
Data Layer — Profiles, Subjects, Attendance, Marks, GPA/CGPA, Notifications
              ↘ Backup / Restore and Operational Logs
```

5. Main Data Flow

1. A user signs in and receives an authenticated session.
2. The user requests an operation permitted for their role.
3. The application validates the request and checks authorization.
4. Business logic reads or updates data through controlled database operations.
5. The application returns the result or a safe error message.
6. Valid changes are stored consistently and covered by the backup process.

6. Requirement Mapping

Attendance and attendance percentage: attendance module and calculation service.
Marks, validation, semester GPA and CGPA: marks/validation and academic calculation services.
Academic records, search and marks summary: record-query services with access checks.
Subject details and credits: subject-management service.
Reports, notifications and downloads: authorized reporting/export services.
Performance, availability, backup and integrity: efficient queries, error handling, backup/restore, validation, and integrity checks.

7. Design Principles

- Keep presentation, business rules, and data access separate.
- Enforce authorization and validation on the server for every protected operation.
- Use institution-approved grading and CGPA formulas; do not invent unconfirmed rules.
- Do not expose credentials or internal error details to users.
- Make calculations and data access independently testable.
- Confirm the final technology stack, API style, hosting, and database with the team.